

Aula 3 Parte 2

August 17, 2026

1 Aula 3 - Bloco 2 (Prática): Estatística, Espaços Vetoriais e Métricas de Distância

Objetivos Práticos do Laboratório:

1. Extrair medidas de **Tendência Central** (Média, Mediana e **Moda**) e **Dispersão** (Variância, Desvio Padrão).
2. Investigar a **Associação** entre atributos usando a **Covariância** e a **Correlação de Pearson**.
3. Replicar a Álgebra Linear no NumPy provando e implementando **from scratch** a Matriz de Covariância.
4. Vetorizar “do zero” funções matemáticas em NumPy para as métricas da IA: **Euclidiana**, **Manhattan**, **Minkowski**, **Cosseno** e **Mahalanobis**.
5. Diagnosticar e solucionar graficamente o **Problema de Escala nos Eixos** através da **Padronização Z-Score**.

```
[1]: # =====
# 1. SETUP DO AMBIENTE E BIBLIOTECAS
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Ajustes gráficos profissionais
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.
    <available else 'default')
plt.rcParams['figure.figsize'] = (10, 6)
pd.set_option('display.float_format', lambda x: '%.4f' % x)

print("=== AMBIENTE CONFIGURADO COM SUCESSO ===")
print(f"NumPy Version : {np.__version__}")
print(f"Pandas Version: {pd.__version__}")

=== AMBIENTE CONFIGURADO COM SUCESSO ===
NumPy Version : 1.24.2
Pandas Version: 1.5.3
```

1.1 1. Construção Numérica e Estatística Básica

Para avaliar medidas centrais, de dispersão e moda, precisaremos de um *dataset* sintético de N=300 clientes bancários contendo variáveis contínuas rigorosamente covariadas, além de dados categóricos discretos essenciais para a verificação da **Moda**.

```
[2]: # =====  
# 2. DATASET SINTÉTICO (VARIÁVEIS CONTÍNUAS E DISCRETAS/CATEGÓRICAS)  
# =====  
np.random.seed(42) # Garantia estatística de reprodutibilidade  
n = 300  
  
# 1. Variável Contínua Independente: Idade (anos)  
idade = np.random.normal(loc=38, scale=12, size=n)  
  
# 2. Variável Contínua Dependente: Renda Mensal (correlação positiva + ruído)  
renda = (idade * 150) + np.random.normal(loc=3500, scale=1800, size=n)  
  
# 3. Variável Contínua Dependente: Score de Crédito (correlação negativa +  
↳ ruído)  
score = 850 - (idade * 4) + np.random.normal(loc=0, scale=30, size=n)  
  
# 4. Variável Categórica/Discreta: Escolaridade e Dependentes  
escolaridade = np.random.choice(  
    ['Ensino Médio', 'Graduação', 'Pós-Graduação', 'Mestrado/Doutorado'],  
    size=n, p=[0.25, 0.45, 0.20, 0.10]  
)  
  
# Estruturando no Pandas  
df = pd.DataFrame({  
    'Idade': idade,  
    'Renda': renda,  
    'Score': score,  
    'Escolaridade': escolaridade  
})  
  
print("--- PRIMEIRAS LINHAS DA BASE DE DADOS ---")  
display(df.head())
```

--- PRIMEIRAS LINHAS DA BASE DE DADOS ---

	Idade	Renda	Score	Escolaridade
0	43.9606	8601.8945	696.8674	Ensino Médio
1	36.3408	7942.7984	676.9717	Pós-Graduação
2	45.7723	11710.9679	692.9991	Pós-Graduação
3	56.2764	13040.1202	665.5637	Graduação
4	35.1902	8740.9011	721.6424	Ensino Médio

1.1.1 1.1 Avaliação das Medidas de Tendência Central, Dispersão e Moda

Modelos precisam entender o centro (μ), a variabilidade matemática (σ) e a **Moda**, sendo esta última crucial para lidar com dados vazios (NaN) nas colunas de texto (Strings).

```
[3]: # =====
# 3. EXTRAÇÃO: MÉDIA, MEDIANA, MODA, VARIÂNCIA E DESVIO PADRÃO
# =====
print("=== MEDIDAS DE TENDÊNCIA CENTRAL ===")
print(f"Média da Renda Mensal      : R$ {df['Renda'].mean():.2f}")
print(f"Mediana da Renda Mensal    : R$ {df['Renda'].median():.2f}")

# A Moda é extraída usando .mode()[0], pois pode haver valores empatados em
# série bimodal.
moda_escolar = df['Escolaridade'].mode()[0]
print(f"\nModa da Escolaridade      : '{moda_escolar}'")
print("-> USO PRÁTICO (Eng. Dados) : Se um cliente não preencheu o campo
      'Escolaridade', "
      "podemos utilizar a Moda para imputar este valor nulo (preenchimento
      seguro).")

print("\n=== MEDIDAS DE DISPERSÃO E ESPALHAMENTO ===")
# O parâmetro (ddof=1) instrui o Pandas a usar a correção de graus de liberdade
# n-1 (Amostral)
var_score = df['Score'].var(ddof=1)
std_score = df['Score'].std(ddof=1)
print(f"Variância do Score (s²)      : {var_score:.2f} pontos²")
print(f"Desvio Padrão do Score (s)    : {std_score:.2f} pontos de afastamento
      médio em relação à base")
```

=== MEDIDAS DE TENDÊNCIA CENTRAL ===

Média da Renda Mensal : R\$ 9151.33

Mediana da Renda Mensal : R\$ 9078.60

Moda da Escolaridade : 'Graduação'

-> USO PRÁTICO (Eng. Dados) : Se um cliente não preencheu o campo
'Escolaridade', podemos utilizar a Moda para imputar este valor nulo
(preenchimento seguro).

=== MEDIDAS DE DISPERSÃO E ESPALHAMENTO ===

Variância do Score (s²) : 3232.62 pontos²

Desvio Padrão do Score (s) : 56.86 pontos de afastamento médio em relação à base

1.2 2. Medidas de Associação e Matrizes Geométricas

A **Covariância** mede a flutuação direcional, mas seus valores são ininterpretáveis fora da grandeza original. A **Correlação de Pearson** transforma as grandezas em um índice normalizado entre $[-1, 1]$. Abaixo o cálculo de forma nativa e, graficamente, a validação via matriz de dispersão.

```
[4]: # =====
# 4. ASSOCIAÇÃO: COVARIÂNCIA E CORRELAÇÃO DE PEARSON
# =====
# Separar a base numérica
df_num = df[['Idade', 'Renda', 'Score']]

print("=== MATRIZ DE COVARIÂNCIA BRUTA (DF.COV) ===")
# Note as escalas mistas e inflacionadas na matriz
display(df_num.cov())

print("\n=== MATRIZ DE CORRELAÇÃO NORMALIZADA DE PEARSON (DF.CORR) ===")
display(df_num.corr())

# Visualização da Associação
plt.figure(figsize=(9, 6))
sns.pairplot(df_num, kind='reg', diag_kind='kde',
              plot_kws={'line_kws':{'color':'crimson', 'lw': 2}, 'scatter_kws':{
↳ {'alpha': 0.5}}})
plt.suptitle("Matriz de Dispersão e Covariância das Variáveis", y=1.02,
↳ fontsize=14, weight='bold')
plt.show()
```

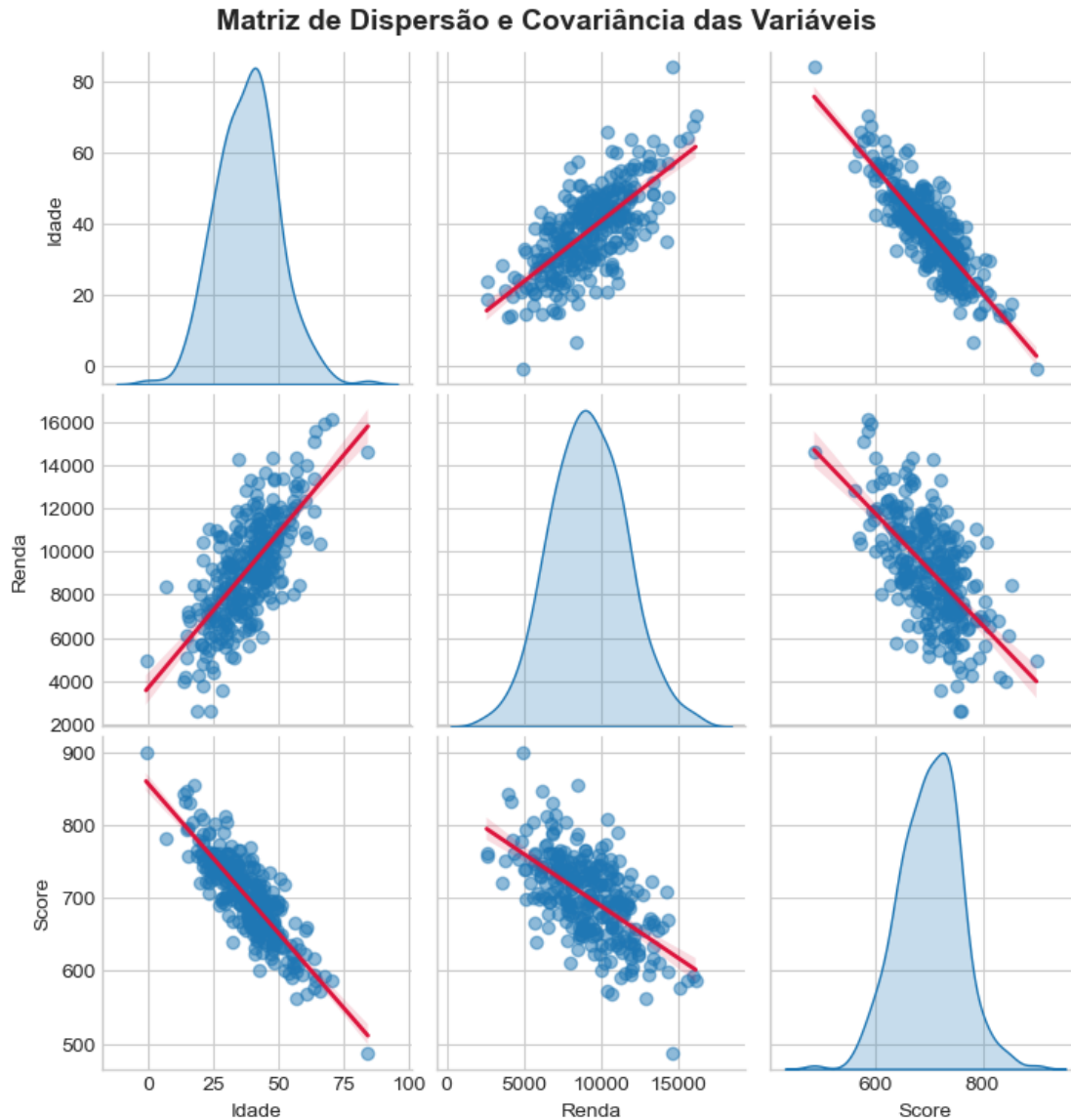
=== MATRIZ DE COVARIÂNCIA BRUTA (DF.COV) ===

	Idade	Renda	Score
Idade	139.4838	20094.5397	-571.2914
Renda	20094.5397	5886034.1225	-84001.7827
Score	-571.2914	-84001.7827	3232.6184

=== MATRIZ DE CORRELAÇÃO NORMALIZADA DE PEARSON (DF.CORR) ===

	Idade	Renda	Score
Idade	1.0000	0.7013	-0.8508
Renda	0.7013	1.0000	-0.6090
Score	-0.8508	-0.6090	1.0000

<Figure size 900x600 with 0 Axes>



1.2.1 Construindo a Matriz de Covariância “From Scratch” em Álgebra Linear (NumPy)

Como vimos na teoria, a arquitetura interna do Pandas baseia-se na multiplicação de tensores matemáticos da seguinte fórmula centralizada populacional:

$$= \frac{1}{n-1} \mathbf{X}_c^T \mathbf{X}_c$$

Onde $\mathbf{X}_c = (\mathbf{X} - \mu)$.

```
[5]: # =====
# 5. COVARIÂNCIA MATRICIAL (FROM SCRATCH EM NUMPY)
```

```
# =====
# 1. Isolar dados puramente como array NumPy
X = df_num.values
n_linhas, p_atributos = X.shape

# 2. Vetorização: Calcular a média ao longo das colunas (eixo 0)
vetor_medias = np.mean(X, axis=0)

# 3. Centralização matricial nativa (broadcasting)
X_centralizado = X - vetor_medias

# 4. Produto Tensorial escalar para a Matriz Sigma (com correção de Bessel n-1)
matriz_sigma_manual = (X_centralizado.T @ X_centralizado) / (n_linhas - 1)

df_sigma = pd.DataFrame(matriz_sigma_manual, index=df_num.columns,
    ↪columns=df_num.columns)
print("=== MATRIZ DE COVARIÂNCIA OBTIDA MANUALMENTE VIA ÁLGEBRA NUMPY ===")
display(df_sigma)

# Prova rigorosa de assertividade computacional (Verifica se são idênticos)
assert np.allclose(matriz_sigma_manual, np.cov(X, rowvar=False)), "Divergência_
    ↪matemática detectada."
print("\n Validação do Cérebro Matricial: Operações de matriz manual coincidem_
    ↪perfeitamente com os cálculos do Numpy/Pandas!")
```

```
=== MATRIZ DE COVARIÂNCIA OBTIDA MANUALMENTE VIA ÁLGEBRA NUMPY ===
```

	Idade	Renda	Score
Idade	139.4838	20094.5397	-571.2914
Renda	20094.5397	5886034.1225	-84001.7827
Score	-571.2914	-84001.7827	3232.6184

Validação do Cérebro Matricial: Operações de matriz manual coincidem
perfeitamente com os cálculos do Numpy/Pandas!

1.3 3. As Distâncias Geométricas Espaciais do Machine Learning

A distância converte observações de banco de dados em conexões algorítmicas lógicas.

Sejam dois clientes **u** e **v** da base. Iremos construir vetorialmente:

- **Euclidiana** (L_2): $\sqrt{\sum (u_k - v_k)^2}$
- **Manhattan** (L_1): $\sum |u_k - v_k|$
- **Cosseno** ($\cos \theta$): $\frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$
- **Mahalanobis**: Baseia-se no espalhamento amostral (requer Σ^{-1})

```
[6]: # =====
# 6. MÉTRICAS DE DISTÂNCIA VETORIZADAS
# =====

def dist_euclidiana(u, v):
    return np.sqrt(np.sum((u - v) ** 2))

def dist_manhattan(u, v):
    return np.sum(np.abs(u - v))

def dist_minkowski(u, v, q=3):
    return np.sum(np.abs(u - v) ** q) ** (1.0 / q)

def similaridade_cosseno(u, v):
    norma_u, norma_v = np.linalg.norm(u), np.linalg.norm(v)
    return np.dot(u, v) / (norma_u * norma_v) if norma_u * norma_v != 0 else 0

def dist_mahalanobis(u, v, sigma_matrix):
    inv_sigma = np.linalg.inv(sigma_matrix)
    delta = u - v
    return np.sqrt(delta.T @ inv_sigma @ delta)

# Testando na prática (Cliente ID 0 vs Cliente ID 1)
cliente_0 = X[0]
cliente_1 = X[1]

print("=== DISTÂNCIAS ENTRE O CLIENTE '0' E CLIENTE '1' NO HIPERESPAÇO ===")
print(f"Euclidiana (L2)           : {dist_euclidiana(cliente_0, cliente_1):.4f}")
print(f"Manhattan (L1)             : {dist_manhattan(cliente_0, cliente_1):.4f}")
print(f"Minkowski (L3)              : {dist_minkowski(cliente_0, cliente_1, q=3):.4f}")
print(f"Similaridade de Cosseno : {similaridade_cosseno(cliente_0, cliente_1):.6f} (A proximidade de 1 indica ângulo similar)")
print(f"Distância Mahalanobis      : {dist_mahalanobis(cliente_0, cliente_1, sigma_matrix_sigma_manual):.4f} desvios estatísticos")
```

```
=== DISTÂNCIAS ENTRE O CLIENTE '0' E CLIENTE '1' NO HIPERESPAÇO ===
Euclidiana (L2)           : 659.4403
Manhattan (L1)            : 686.6115
Minkowski (L3)            : 659.1025
Similaridade de Cosseno   : 0.999991 (A proximidade de 1 indica ângulo similar)
Distância Mahalanobis     : 1.8387 desvios estatísticos
```

1.4 1. Distância de Hamming

A distância de Hamming mede o número de posições diferentes entre duas sequências de **mesmo comprimento**.

```
[1]: def hamming_distance(seq1, seq2):
    """
    Calcula a Distância de Hamming entre duas strings ou listas.
    As sequências devem ter o mesmo comprimento.
    """
    # Validação obrigatória: comprimentos idênticos
    if len(seq1) != len(seq2):
        raise ValueError("As sequências devem ter exatamente o mesmo_
↪comprimento.")

    # Conta as posições onde os elementos são diferentes
    return sum(el1 != el2 for el1, el2 in zip(seq1, seq2))

# Teste da Distância de Hamming
codigo_a = "1011101"
codigo_b = "1001001"

dist_hamming = hamming_distance(codigo_a, codigo_b)
print(f"Palavra A: {codigo_a}")
print(f"Palavra B: {codigo_b}")
print(f"Distância de Hamming: {dist_hamming}\n")
```

```
Palavra A: 1011101
Palavra B: 1001001
Distância de Hamming: 2
```

1.5 2. Distância de Jaccard

A distância de Jaccard mede a dissimilaridade entre dois conjuntos. Ela é calculada como $1 - \text{Índice de Jaccard}$, onde o índice é a interseção dividida pela união dos conjuntos.

```
[2]: def jaccard_distance(seq1, seq2):
    """
    Calcula a Distância de Jaccard entre duas coleções.
    Aceita strings, listas ou conjuntos e não exige comprimentos iguais.
    """
    # Conversão para conjuntos (remove duplicadas automaticamente)
    set1 = set(seq1)
    set2 = set(seq2)

    # Tratamento para conjuntos vazios (evita divisão por zero)
    if not set1 and not set2:
        return 0.0

    # Cálculo da Interseção e União
    intersecao = len(set1.intersection(set2))
    uniao = len(set1.union(set2))
```

```

# Índice de Jaccard (similaridade)
indice_jaccard = intersecao / uniao

# Distância de Jaccard (dissimilaridade)
return 1 - indice_jaccard

# Teste da Distância de Jaccard
texto_a = "dados"
texto_b = "dadosfera"

dist_jaccard = jaccard_distance(texto_a, texto_b)
print(f"Texto A: '{texto_a}' -> Conjunto: {set(texto_a)}")
print(f"Texto B: '{texto_b}' -> Conjunto: {set(texto_b)}")
print(f"Distância de Jaccard: {dist_jaccard:.4f}")

```

Texto A: 'dados' -> Conjunto: {'d', 'a', 's', 'o'}
 Texto B: 'dadosfera' -> Conjunto: {'a', 'e', 'f', 'o', 'd', 'r', 's'}
 Distância de Jaccard: 0.4286

1.6 4. O Impacto Estrutural da Padronização Z-Score

Note que as distâncias espaciais físicas (Euclidiana) atingiram números altíssimos (>2000). Isso prova o erro fundamental da “cegueira do algoritmo para unidades de medida”. O eixo Y de **Renda** ofusca inteiramente as variações ocorridas no eixo X da **Idade**.

O Escalonamento **Z-Score** redimensiona os vetores forçando todas as colunas para o centro universal: Média 0 e Variância 1.

$$z = \frac{x - \mu}{\sigma}$$

```

[7]: # =====
# 7. PADRONIZAÇÃO (Z-SCORE) E SUA IMPORTÂNCIA ESTRUTURAL GRÁFICA
# =====
# Extrair Desvios Padrão vetoriais das colunas
desvios_colunas = np.std(X, axis=0, ddof=1)

# Aplicar o Z-score manual aproveitando as matrizes numpy (X_centralizado já é
↳ X - )
X_padronizado_Z = X_centralizado / desvios_colunas

# Verificar distâncias escalonadas agora (Comparação justa e equilibrada)
cliente_0_z = X_padronizado_Z[0]
cliente_1_z = X_padronizado_Z[1]
dist_justa_z = dist_euclidiana(cliente_0_z, cliente_1_z)

print("=== COMPARAÇÃO ESTRUTURAL APÓS O Z-SCORE ===")
print(f"Antiga Distância Euclidiana Errada (Viesada na Renda) :↳
↳ {dist_euclidiana(cliente_0, cliente_1):.4f}")

```

```

print(f"Nova Distância Euclidiana Z-Score (Justa p/ Atributos): {dist_justa_z:.
    ↪4f}")

# Plotagem lado a lado do fenômeno
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot Espaço Bruto
ax1.scatter(X[:, 0], X[:, 1], color='maroon', alpha=0.7, edgecolor='k')
ax1.set_title("O Risco: Escalas Dispares Sem Z-Score\nEixo Y cresce milhares de
    ↪vezes frente a X", pad=15, weight='bold')
ax1.set_xlabel("Eixo X (Idade em Anos)")
ax1.set_ylabel("Eixo Y (Renda Mensal R$)")

# Plot Espaço Corrigido Z-Score
ax2.scatter(X_padronizado_Z[:, 0], X_padronizado_Z[:, 1], color='darkgreen',
    ↪alpha=0.7, edgecolor='k')
ax2.set_title("O Acerto: Espaço Vetorial com Z-Score (Equilibrado)\nEscala
    ↪Paramétrica Universal", pad=15, weight='bold')
ax2.set_xlabel("Idade Transformada (z)")
ax2.set_ylabel("Renda Mensal Transformada (z)")
ax2.axhline(0, color='gray', linestyle='--', alpha=0.6)
ax2.axvline(0, color='gray', linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()

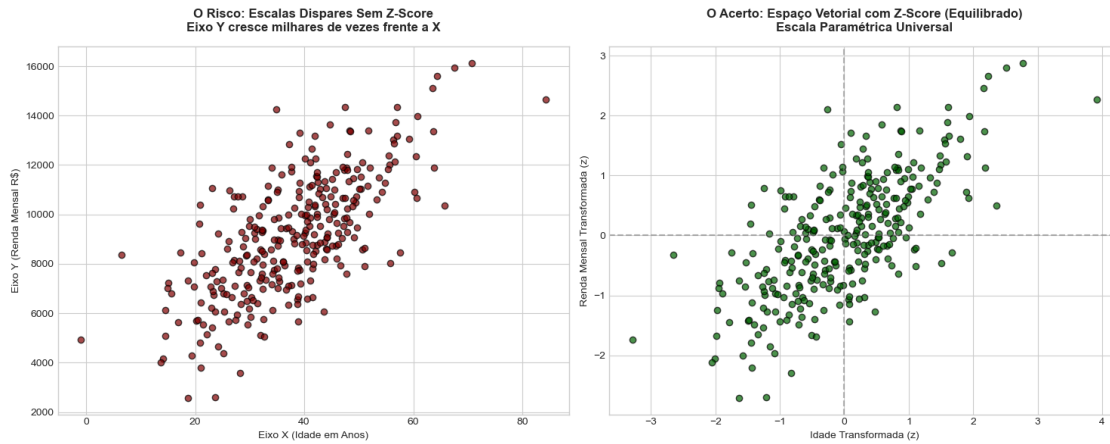
print("== CONCLUSÃO DA AULA PRÁTICA ==")
print("A Geometria dos Dados exige que todos os eixos obedecam à mesma ordem de
    ↪grandeza.")
print("Sem padronização estatística, algoritmos puramente espaciais como o
    ↪K-Means ou SVM colapsarão e emitirão classificações imprecisas baseando-se
    ↪em ruído de escala.")

```

=== COMPARAÇÃO ESTRUTURAL APÓS O Z-SCORE ===

Antiga Distância Euclidiana Errada (Viesada na Renda) : 659.4403

Nova Distância Euclidiana Z-Score (Justa p/ Atributos): 0.7826



== CONCLUSÃO DA AULA PRÁTICA ==

A Geometria dos Dados exige que todos os eixos obedeçam à mesma ordem de grandeza.

Sem padronização estatística, algoritmos puramente espaciais como o K-Means ou SVM colapsarão e emitirão classificações imprecisas baseando-se em ruído de escala.

[]: